

VIT BHOPAL UNIVERSITY

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

PROJECT REPORT

STUDENT INFORMATION SYSTEM

Programming in Java

Submitted By	Tanmay Vikas Madhekar
Register Number	25BAI10377
Program / Branch	CSE (AI & ML)
Semester	Fall Semester
Faculty	Jay Prakash Maurya
Academic Year	2026–27

Academic Year 2026–27

1. Introduction

The **Student Information System** is a Java-based terminal application developed to manage student records in a simple, structured, and reliable manner. The application provides a menu-driven console interface through which the user can add, display, search, update, and delete student information.

The implementation uses Core Java concepts including classes and objects, encapsulation, ArrayList, methods, loops, conditional statements, exception handling, file handling, and object serialization. Student records are stored locally in **students.dat** so that saved information can be restored when the application is started again.

2. Problem Statement

Manual management of student records can become difficult as the number of records increases. It may result in duplicate IDs, incorrect GPA values, difficulty locating a student, and loss of information. The proposed system provides a terminal-based solution for maintaining student ID, name, department, and GPA with validation and persistent local storage.

3. Functional Requirements

ID	Requirement
FR-01	Add a new student record.
FR-02	Prevent duplicate student IDs.
FR-03	Display all registered students.
FR-04	Search a student using ID.
FR-05	Update name, department, and optionally GPA.
FR-06	Delete a student after confirmation.
FR-07	Accept GPA only from 0.0 to 4.0.
FR-08	Handle invalid integer and decimal input.
FR-09	Load saved records from students.dat.
FR-10	Save records using Java serialization when exiting.

4. Non-functional Requirements

- **Usability:** Clear menu-driven terminal interaction.
- **Reliability:** Invalid input is handled using validation and exception handling.
- **Maintainability:** Operations are separated into methods.
- **Performance:** Suitable response time for a small student dataset.
- **Portability:** Runs on systems with a compatible Java JDK.
- **Persistence:** Records remain available after saving and restarting the application.

5. System Architecture

The application follows a simple console-based architecture consisting of user interaction, application logic, in-memory data, and file storage.

USER
Terminal / Console Interface
StudentManagementSystem Menu • CRUD Operations • Search • Validation
ArrayList<Student> In-memory student records
students.dat Java Object Serialization

Architecture flow: User → Console Menu → StudentManagementSystem → ArrayList<Student> → students.dat

6. Design Diagrams

6.1 Use Case Diagram

ACTOR	USE CASES
User	Add Student Display All Students Search Student by ID Update Student Record Delete Student Record Save & Exit

The user is the primary actor and can perform all major system operations through the terminal.

6.2 Workflow Diagram

START
loadData()
Display Main Menu
Read option 1–6
Perform selected operation
Return to menu
Option 6 → saveData()
END

6.3 Sequence Diagram – Add Student

User	System	Student	File
Select Add Student	Request ID/name/department/GPA		
	findStudentById()		
	Create Student object	Object created	
	students.add()	Record stored in ArrayList	
	saveData() on exit		students.dat

6.4 Class / Component Diagram

Student ----- - id : String - name : String - department : String - gpa : double ----- + getters / setters + toString()	StudentManagementSystem ----- - FILE_NAME : String - students : List<Student> - scanner : Scanner ----- + main() + printMenu() + addStudent() + viewAllStudents() + searchStudent() + updateStudent() + deleteStudent() + loadData() + saveData()
---	---

Relationship: StudentManagementSystem maintains a collection of Student objects using ArrayList<Student>.

6.5 ER Diagram / Data Model

The project does not use a relational database. It stores serialized Student objects. The logical data model is represented by the following Student entity:

STUDENT
id — unique student identifier
name — full name
department — academic department
gpa — value from 0.0 to 4.0
Persistence: students.dat

7. Design Decisions & Rationale

Decision	Rationale
Terminal interface	Keeps the project focused on Java fundamentals without GUI complexity.
ArrayList<Student>	Provides simple dynamic storage for a small dataset.
Serializable	Allows Student objects and the list to be stored and restored.
students.dat	Provides local persistence without an external database.
Case-insensitive ID search	Makes student ID lookup more convenient.
GPA validation	Ensures GPA remains within the accepted 0.0–4.0 range.
Separate methods	Improves readability, modularity, and maintainability.

8. Implementation Details

Student class: Implements Serializable and stores four private fields: id, name, department, and gpa. A constructor initializes each object. Getter and setter methods provide controlled access, while toString() formats a record for terminal display.

StudentManagementSystem class: Maintains an ArrayList of Student objects and a Scanner. The main() method loads saved data, displays the menu, reads the user's choice, and invokes the selected operation.

CRUD operations: addStudent() creates a record after checking duplicate IDs; viewAllStudents() displays records; searchStudent() finds a record by ID; updateStudent() changes selected fields; deleteStudent() removes a record after confirmation.

Input validation: readIntInput() repeatedly requests valid integers. readDoubleInput() accepts only numeric GPA values from 0.0 to 4.0.

Persistence: loadData() reads the serialized list using ObjectInputStream. saveData() writes the list using ObjectOutputStream when the user chooses Save & Exit.

9. Screenshots / Results

The following screenshots were captured from the actual Java application running in the VS Code environment. They demonstrate record creation, display, update, deletion, and terminal interaction.

Figure 1. Adding and displaying student records.

```

/usr/bin/env /Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/tanmayvikasmadhekar/...
StudentManagementSystem

=====
STUDENT INFORMATION SYSTEM
=====
1. Add Student
2. Display All Students
3. Search Student by ID
4. Update Student Record
5. Delete Student Record
6. Save & Exit
=====
Enter option (1-6): 1

--- Add New Record ---
Enter ID: 25BA110377
Enter Full Name: Tanmay VM
Enter Department: CSE
Enter GPA (0.0 - 4.0): 3.9
Student record added successfully.

=====
STUDENT INFORMATION SYSTEM
=====
1. Add Student
2. Display All Students
3. Search Student by ID
4. Update Student Record
5. Delete Student Record
6. Save & Exit
=====
Enter option (1-6): 4

Enter Student ID to update: 25BA110377
Current Record: | 25BA110377 | Tanmay VM | CSE | 3.90 |
New Name (press Enter to skip):
New Department (press Enter to skip):
Update GPA? (y/n): 4.0
Record updated successfully.

=====
STUDENT INFORMATION SYSTEM
=====
1. Add Student
2. Display All Students
3. Search Student by ID
4. Update Student Record
5. Delete Student Record
6. Save & Exit
=====
Enter option (1-6): 2

--- Registered Students ---
| ID | Name | Department | GPA |
|----|-----|-----|----|

```

Figure 2. Displaying multiple registered students.

```
=====
Enter option (1-6): 2
----- Registered Students -----
| ID          | Name      | Department | GPA  |
|-----|-----|-----|-----|
| 25BA110377  | Tanmay VM | CSE        | 3.90 |
=====

STUDENT INFORMATION SYSTEM
=====
1. Add Student
2. Display All Students
3. Search Student by ID
4. Update Student Record
5. Delete Student Record
6. Save & Exit
=====
Enter option (1-6): 1

----- Add New Record -----
Enter ID: 25BCY10191
Enter Full Name: Utkarsh
Enter Departments: CSE
Enter GPA (0.0 - 4.0): 3.7
Student record added successfully.

STUDENT INFORMATION SYSTEM
=====
1. Add Student
2. Display All Students
3. Search Student by ID
4. Update Student Record
5. Delete Student Record
6. Save & Exit
=====
Enter option (1-6): 2
----- Registered Students -----
| ID          | Name      | Department | GPA  |
|-----|-----|-----|-----|
| 25BA110377  | Tanmay VM | CSE        | 3.90 |
| 25BCY10191  | Utkarsh   | CSE        | 3.70 |
=====

STUDENT INFORMATION SYSTEM
=====
1. Add Student
2. Display All Students
3. Search Student by ID
4. Update Student Record
5. Delete Student Record
6. Save & Exit
=====
```

Figure 3. Updating and displaying a student record.

```
5. Delete Student Record
6. Save & Exit
=====
Enter option (1-6): 2

--- Registered Students ---


| ID         | Name      | Department | GPA  |
|------------|-----------|------------|------|
| 25BA110377 | Tanmay VM | CSE        | 3.90 |
| 25BCY10191 | Utkarsh   | CSE        | 3.70 |



=====
STUDENT INFORMATION SYSTEM
=====
1. Add Student
2. Display All Students
3. Search Student by ID
4. Update Student Record
5. Delete Student Record
6. Save & Exit
=====
Enter option (1-6): 5

Enter Student ID to delete: 25BA110377
Confirm deletion of Tanmay VM? (y/n): y
Record deleted successfully.

=====
STUDENT INFORMATION SYSTEM
=====
1. Add Student
2. Display All Students
3. Search Student by ID
4. Update Student Record
5. Delete Student Record
6. Save & Exit
=====
Enter option (1-6): 2

--- Registered Students ---


| ID         | Name    | Department | GPA  |
|------------|---------|------------|------|
| 25BCY10191 | Utkarsh | CSE        | 3.70 |



=====
STUDENT INFORMATION SYSTEM
=====
1. Add Student
2. Display All Students
3. Search Student by ID
4. Update Student Record
5. Delete Student Record
6. Save & Exit
=====
Enter option (1-6):
```

10. Testing Approach

Testing focuses on normal operations, invalid inputs, boundary values, and data persistence. The supplied screenshots also demonstrate successful execution of major CRUD operations.

TC	Test Input / Action	Expected Result
TC-01	Valid new student	Record added successfully.
TC-02	Existing student ID	Duplicate ID rejected.
TC-03	GPA below 0 or above 4	Value rejected and input requested again.
TC-04	Display all students	All records displayed.
TC-05	Existing ID search	Matching record displayed.
TC-06	Unknown ID search	No record found message.
TC-07	Update existing record	Selected fields updated.
TC-08	Delete + y confirmation	Record deleted.
TC-09	Delete + n confirmation	Deletion cancelled.
TC-10	Restart after Save & Exit	Saved records loaded from students.dat.
TC-11	Non-numeric menu input	Input error shown; program continues.

11. Challenges Faced

- Handling invalid numeric input while keeping the application running.
- Preventing duplicate student IDs and locating records reliably.
- Updating and deleting objects inside an ArrayList.
- Implementing Java object serialization and deserialization.
- Maintaining readable table formatting in the terminal.
- Designing a menu that supports all required operations clearly.

12. Learnings & Key Takeaways

- Applied object-oriented programming concepts in a complete Java application.
- Used encapsulation through private fields and getter/setter methods.
- Used ArrayList for dynamic collections of objects.
- Applied exception handling and input validation.
- Implemented file handling using ObjectInputStream and ObjectOutputStream.
- Understood CRUD operations and basic data persistence.
- Learned how to organize and document a Java project for GitHub.

13. Future Enhancements

- Add login and role-based access for administrators and teachers.
- Add phone number, email, semester, and course details.
- Use MySQL or another database instead of serialized files.
- Add sorting and filtering by department or GPA.
- Export records to CSV or PDF.
- Develop a GUI or web version.
- Add automated unit testing using JUnit.

14. References

- Oracle Java Documentation – Java language and API documentation.
- Java SE documentation – Collections Framework, I/O, and Serialization.
- Programming in Java course material, VIT Bhopal University.
- Java Development Kit (JDK) documentation.

15. Conclusion

The Student Information System demonstrates how Core Java can be used to develop a practical terminal-based record-management application. The implemented system supports CRUD operations, input validation, case-insensitive student ID search, and persistent local storage through Java serialization. The project provides a clear foundation for future improvements such as database integration, authentication, automated testing, and a graphical or web interface.